

Pointers and its concepts

From variables to their addresses



Basics of pointers

Basic Concept

In memory, every stored data item occupies one or more contiguous memory cells.

- **The number of memory cells required to store a data item depends on its type (char, int, double, etc.).**

Whenever we declare a variable, the system allocates memory location(s) to hold the value of the variable.

- **Since every byte in memory has a unique address, this location will also have its own (unique) address.**

Accessing the Address of a Variable

The address of a variable can be determined using the ‘&’ operator.

- The operator ‘&’ immediately preceding a variable returns the *address* of the variable
- & is the “*address-of*” operator

Example: `&xyz`

The ‘&’ operator can be used only with a *simple variable* or an *array element*.

`&distance`

`&x[0]`

Following usages are *illegal*:

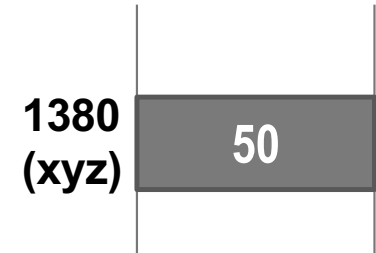
`&235` – address of a constant is not defined

`&(a+b)` – address of an expression is not defined

Example

Consider the statement

```
int xyz = 50;
```



- This statement instructs the compiler to allocate a location for the integer variable **xyz**, and put the value **50** in that location.
- Suppose that the (starting) address location chosen is **1380**.
- During execution of the program, the system always associates the name **xyz** with the address **1380**.
- The value **50** can be accessed by using either the name (**xyz**) or the address (**&xyz** which equals **1380** in this example).

Pointer Declaration

A pointer is just a C variable whose **value** is the **address** of another variable!

Pointer variables must be declared before we use them.

General form:

```
data_type *pointer_name;
```

Example:

```
int *ptr;
```

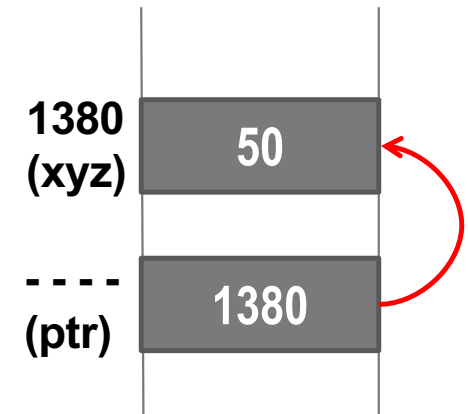
Three things are specified in the above declaration:

- The asterisk (*) tells that the variable **pointer_name** is a pointer variable.
- **pointer_name** will be used to points to a variable of type **data_type**.

Just after declaring a pointer, ptr does not actually point to anything yet (remember: a pointer is also a variable; hence can contain garbage until it is assigned to some specific value)

Example (Contd.)

```
int xyz = 50;  
int *ptr; // Here ptr is a pointer to an integer  
ptr = &xyz;
```



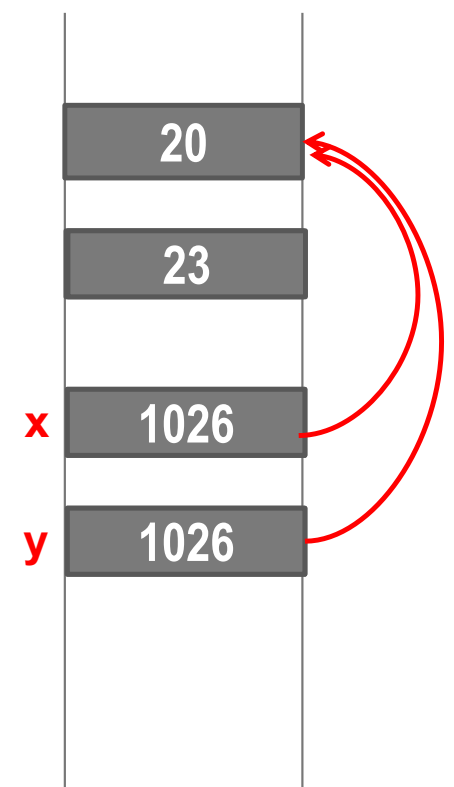
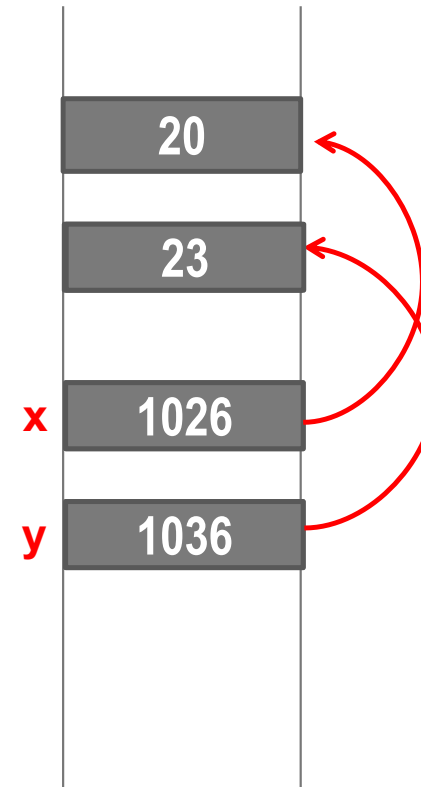
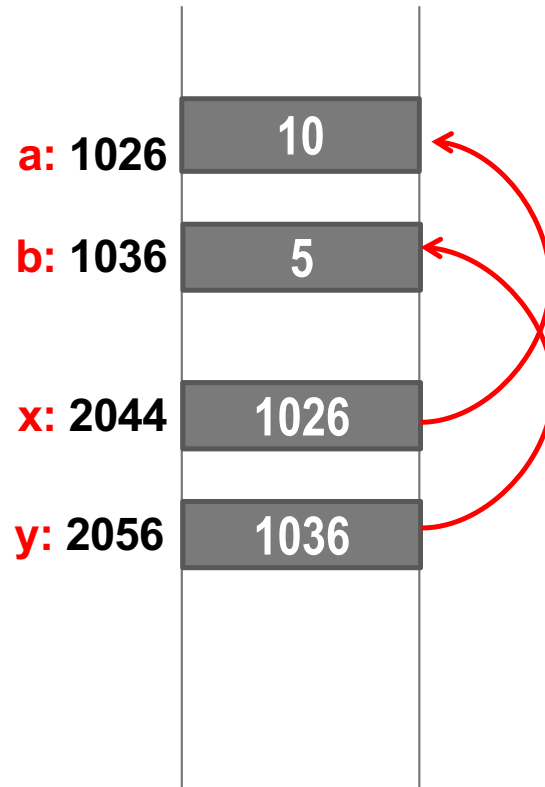
Since memory addresses are simply numbers, they can be assigned to some variables which can be stored in memory.

- Such variables that hold memory addresses are called **pointers**.
- Since a pointer is a variable, its value is also stored in some memory location.

Once ptr has been assigned a valid memory address, the * operator can be used to access the value at that address. *** is the “value-at” operator; can be used only with a pointer variable**

Example: Making a pointer point to a variable

```
int a=10, b=5;  
int *x, *y;  
x= &a; y=&b;  
*x= 20;  
*y= *x + 3;  
y= x;
```



Given a pointer variable, we can either:

- make it point to (i.e., store the address of) some existing variable, or
- dynamically allocate memory and make it point to it (to be discussed later)

Things to Remember

Pointers have types, e.g. :

```
int    *count;  
float  *speed;
```

Pointer variables must always point to a data item of the *same type*.

```
float x;  
int  *p;  
p = &x;    // You should not generally do this
```

However, type casting can be used in some circumstances – we will see examples later

Pointers and arrays

Pointers and Arrays

When an array is declared:

- The array has a **base address** and sufficient amount of storage to contain all the elements of the array in contiguous memory locations.
- The **base address** is the location of the first element (**index 0**) of the array.
- The compiler also defines the **array name as a constant pointer to the first element**.

Example

Consider the declaration:

```
int x[5] = {1, 2, 3, 4, 5};
```

```
int* p;
```

- Suppose that the base address of x is 2500, and each integer requires 4 bytes.

<u>Element</u>	<u>Value</u>	<u>Address</u>
x[0]	1	2500
x[1]	2	2504
x[2]	3	2508
x[3]	4	2512
x[4]	5	2516

Both **x** and **&x[0]** have the value **2500**.

p = x; and **p = &x[0];** are equivalent

Example (contd)

- Suppose we assign $p = \&x[0];$
- Now we can access successive values of x by using $p++$ or $p--$ to move from one element to another.

Relationship between p and x :

$p = \&x[0] = 2500$

$p+1 = \&x[1] = 2504$

$p+2 = \&x[2] = 2508$

$p+3 = \&x[3] = 2512$

$p+4 = \&x[4] = 2516$

$(p+i)$ gives the address of $x[i]$

$(p+i)$ is the same as $\&x[i]$

$*(p+i)$ gives the value of $x[i]$

Pointer expressions

Pointer arithmetic

Pointer Expressions

Like other variables, pointer variables can be used in expressions.

If p1 and p2 are two pointers, the following statements are valid:

```
sum = (*p1) + (*p2);
```

```
prod = (*p1) * (*p2);
```

```
*p1 = *p1 + 2;
```

```
x = *p1 / *p2 + 5;
```

More on pointer expressions

What are allowed in C?

- Add an integer to a pointer.
- Subtract an integer from a pointer.
- Subtract one pointer from another
 - If $p1$ and $p2$ are both pointers to the same array, then $p2 - p1$ gives the number of elements between $p1$ and $p2$.

More on pointer expressions

What are not allowed?

- Add two pointers.

$p1 = p1 + p2;$

- Multiply / divide a pointer in an expression.

$p1 = p2 / 5;$

$p1 = p1 - p2 * 10;$

Scale Factor

We have seen that an integer value can be added to or subtracted from a pointer variable.

```
int x[ 5 ] = { 10, 20, 30, 40, 50 };  
int *p;
```

```
p = &x[1];  
printf( "%d", *p);           // This will print 20
```

```
p++;                          // This increases p by the number of bytes for an integer  
printf( "%d", *p);           // This will print 30
```

```
p = p + 2;                    // This increases p by twice the sizeof(int)  
printf( "%d", *p);           // This will print 50
```

More on Scale Factor

```
char carr[ 5 ] = { 'A', 'B', 'p', '?', 'S' };  
int darr[ 5 ] = { 10, 20, 30, 40, 50 }  
char *p; int* q;
```

```
p = carr;           // The pointer p now points to the first element of the array  
q = darr;  
p = p + 1;          // Now p points to the second element in the array carr  
q = q + 1;          // Now q points to the second element in the array darr
```

When a pointer variable is increased by 1, the increment is not necessarily by one byte, but by the *size of the data type* to which the pointer points.

This is why we have many data types for pointers, not just a single “address” data type

Pointer types and scale factor

<u>Data Type</u>	<u>Scale Factor</u>
char	1
int	4
float	4
double	8

- If p1 is an integer pointer, then

p1++

will increment the value of **p1** by **4**.

Scale factor may be machine dependent

- The exact scale factor may vary from one machine to another.
- Can be found out using the `sizeof` function.

```
#include <stdio.h>
main( )
{
    printf ("No. of bytes occupied by int is %d \n", sizeof(int));
    printf ("No. of bytes occupied by float is %d \n", sizeof(float));
    printf ("No. of bytes occupied by double is %d \n", sizeof(double));
    printf ("No. of bytes occupied by char is %d \n", sizeof(char));
}
```

Output:

Number of bytes occupied by int is 4

Number of bytes occupied by float is 4

Number of bytes occupied by double is 8

Number of bytes occupied by char is 1

Pointers and functions

Passing Pointers to a Function

In C, arguments are passed to a function *by value*.

- The data items are copied to the function.
- Changes made in the called function are not reflected in the calling function.

Pointers are often passed to a function as arguments.

- Allows data items within the calling function to be accessed by the called function (through their address) and modified

Passing pointers as arguments to functions

```
#include <stdio.h>
main( )
{
    int a, b;
    a = 5; b = 20;
    swap (a, b);
    printf (“\n a=%d, b=%d”, a, b);
}
```

```
void swap (int x, int y)
{
    int t;
    t = x; x = y; y = t;
}
```

Output

a=5, b=20

```
#include <stdio.h>
main( )
{
    int a, b;
    a = 5; b = 20;
    swap (&a, &b);
    printf (“\n a=%d, b=%d”, a, b);
}
```

```
void swap (int *x, int *y)
{
    int t;
    t = *x; *x = *y; *y = t;
}
```

Output

a=20, b=5

A useful application of pointers

In C, a function can only return a single value

Suppose you want to write a function that computes two values. How to send both the computed values back to the calling function (e.g., main function)?

One way:

- Declare variables within the main (calling) function
- Pass addresses of these variables as arguments to the function
- The called function can directly store the computed values in the variables declared within main

Another application of Pointers: Dynamic memory allocation

Problem with arrays

Sometimes

- Amount of data cannot be predicted beforehand (maybe, user input)
- Number of data items keeps changing during program execution

Example: Search for an element in an array of N elements

One solution: assume a maximum possible value of N and allocate array of N elements

- Wastes memory space, as N may be much smaller in some executions
- Example: maximum value of N may be 10,000, but a particular run may need to search only among 100 elements
 - Using array of size 10,000 always wastes memory in most cases

Better solution

Dynamic memory allocation

- Know how much memory is needed after the program is run
 - Example: ask the user to enter from keyboard
- Dynamically allocate only the amount of memory needed

C provides functions to dynamically allocate memory

- malloc, calloc, realloc

Dynamic Memory Allocation

Normally the number of elements in an array is specified in the program

- Often leads to wastage of memory space or program failure.

Dynamic Memory Allocation

- Memory space required can be specified **at the time of execution**.
- C supports allocating and freeing memory dynamically using library routines.

Memory Allocation Functions

`malloc`

- Allocates requested number of bytes and returns a pointer to the first byte of the allocated space.

`calloc`

- Allocates space for an array of elements, initializes them to zero and then returns a pointer to the memory.

`free`

- Frees previously allocated space.

`realloc`

- Modifies the size of previously allocated space.

Allocating a Block of Memory

A block of memory can be allocated using the function **malloc**.

- Reserves a block of memory of specified size and returns a pointer of type **void**.
- The return pointer can be type-casted to any pointer type.

General format:

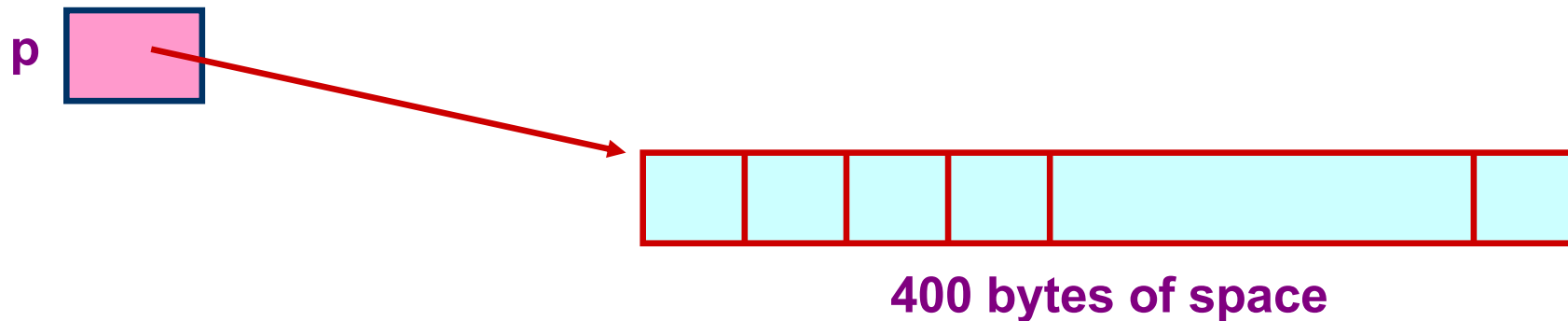
```
ptr = (type *) malloc (byte_size);
```

Allocating a Block of Memory

Examples

```
p = (int *) malloc(100 * sizeof(int));
```

- A memory space equivalent to (*100 times the size of an int*) bytes is reserved.
- The address of the first byte of the allocated memory is assigned to the pointer **p** of type **int**.



Allocating a Block of Memory

```
cptr = (char *) malloc (20);
```

- Allocates 20 bytes of space for the pointer `cptr` of type `char`.

Points to Note

`malloc` always allocates a block of contiguous bytes.

- The allocation can fail if sufficient contiguous memory space is not available.
- If it fails, `malloc` returns `NULL`.

```
if ((p = (int *) malloc(100 * sizeof(int))) == NULL)
{
    printf ("\n Memory cannot be allocated");
    exit( ) ;
}
```

Can we allocate only arrays?

malloc can be used to allocate memory for single variables also

- `p = (int *) malloc (sizeof(int));`
- Allocates space for a single int, which can be accessed as `*p`
 - Single variable allocations are just special case of array allocations
 - **Array with only one element**

Using the malloc'd Array

Once the memory is allocated, it can be used with pointers, or with array notation

Example:

```
int *p, n, i;  
scanf("%d", &n);  
p = (int *) malloc (n * sizeof(int));  
for (i=0; i<n; ++i)  
    scanf("%d", &p[i]);
```

The n integers allocated can be accessed as `*p, *(p+1), *(p+2), ..., *(p+n-1)` or just as `p[0], p[1], p[2], ..., p[n-1]`

Releasing the allocated space: **free**

An allocated block can be returned to the system for future use by using the **free** function

General syntax:

```
free (ptr);
```

where **ptr** is a pointer to a memory block which has been previously created using **malloc**

Note that no size needs to be mentioned for the allocated block, the system remembers it for each pointer returned

Example

```
int main()
{
    int i,N;
    float *height;
    float sum=0,avg;

    printf("Input no. of students\n");
    scanf("%d", &N);

    height = (float *)
        malloc(N * sizeof(float));
```

```
    printf("Input heights for %d
students \n",N);
    for (i=0; i<N; i++)
        scanf ("%f", &height[i]);

    for(i=0;i<N;i++)
        sum += height[i];

    avg = sum / (float) N;

    printf("Average height = %f \n",
        avg);

    free (height);
    return 0;
}
```

Altering the Size of a Block

Sometimes we need to alter the size of some previously allocated memory block.

- More memory needed.
- Memory allocated is larger than necessary.

How?

- By using the `realloc` function.

If the original allocation is done as:

```
ptr = malloc (size);
```

then reallocation of space may be done as:

```
ptr = realloc (ptr, newsize);
```

Altering the Size of a Block

- The new memory block may or may not begin at the same place as the old one.
 - If it does not find space, it will create it in an entirely different region and move the contents of the old block into the new block.
- The function guarantees that the old data remains intact.
- If it is unable to allocate, it returns **NULL** and frees the original block.